EXPERIMENT 04

Finite-State Machine for Morphological Parsing (Noun Pluralization)

AIM

To implement a rule-based finite-state transducer/machine in Python to perform morphological generation of plural forms for English nouns.

PROCEDURE

1. Formulate English pluralization rules including sibilants (-s, -x, -z, -ch, -sh -> +es), consonant + y (-y -> -ies), irregular nouns, and default affixation (+s).
2. Design the state transition logic modeling root morphology to surface inflected form.
3. Implement helper functions to detect terminal character classes (vowels, consonants, sibilants).
4. Process singular nouns through the finite-state machine to generate the target plural forms.
5. Validate outputs with both regular and irregular noun paradigms.

PROGRAM

```
class MorphologicalPluralFSM:
    def __init__(self):
        self.irregulars = {
            "child": "children",
            "foot": "feet",
            "tooth": "teeth",
            "mouse": "mice",
            "person": "people",
            "ox": "oxen",
            "man": "men",
            "woman": "women"
        }

    def pluralize(self, noun):
        word = noun.strip().lower()
        if not word:
            return ""

        if word in self.irregulars:
            return self.irregulars[word]

        if word.endswith(('s', 'x', 'z', 'sh', 'ch')):
            return word + "es"

        vowels = 'aeiou'
        if word.endswith('y') and len(word) > 1 and word[-2] not in vowels:
            return word[:-1] + "ies"

        if word.endswith('fe'):
```

```
            return word[:-2] + "ves"
        elif word.endswith('f') and not word.endswith(('ff', 'oof', 'ief')):
            return word[:-1] + "ves"

        if word.endswith('o') and len(word) > 1 and word[-2] not in vowels:
            return word + "es"

        return word + "s"

fsm = MorphologicalPluralFSM()
test_nouns = ["cat", "dog", "bus", "box", "church", "baby", "boy", "leaf", "knife",
"child", "mouse", "hero"]

print(f"{'Singular':<12} | {'Generated Plural':<16} | {'Rule Applied'}")
print("-" * 50)
for n in test_nouns:
    res = fsm.pluralize(n)
    rule = "Irregular Lexicon" if n in fsm.irregulars else \
           "+es (Sibilant/o)" if res.endswith("es") and not res.endswith("ies") and
not res.endswith("ves") else \
           "-y -> +ies" if res.endswith("ies") else \
           "-f/fe -> +ves" if res.endswith("ves") else "+s (Default)"
    print(f"{n:<12} | {res:<16} | {rule}")
```

OUTPUT

```
> python3 experiment_04.py
--------------------------------------------------------------------------------------------------
Singular     | Generated Plural | Rule Applied
--------------------------------------------------
cat          | cats             | +s (Default)
dog          | dogs             | +s (Default)
bus          | buses            | +es (Sibilant/o)
box          | boxes            | +es (Sibilant/o)
church       | churches         | +es (Sibilant/o)
baby         | babies           | -y -> +ies
boy          | boys             | +s (Default)
leaf         | leaves           | -f/fe -> +ves
knife        | knives           | -f/fe -> +ves
child        | children         | Irregular Lexicon
mouse        | mice             | Irregular Lexicon
hero         | heroes           | +es (Sibilant/o)
[Process completed with exit code 0]
```

RESULT

Thus, a finite-state machine for morphological parsing and plural noun generation was successfully implemented and verified.